Reproducing AlphaZero on Tablut: Self-Play RL for an Asymmetric Board Game

Tõnis Lees
Institute of Computer Science
University of Tartu
tonis.lees@gmail.com

Tambet Matiisen*
Institute of Computer Science
University of Tartu
tambet.matiisen@ut.ee

1 Introduction

Silver et al. [2018] introduced $AlphaZero$, a general reinforcement learning algorithm that masters Chess, Shogi, and Go through self-play with no domain-specific knowledge beyond the rules. A single neural network $(\mathbf{p}, v) = f_\theta(s)$ is trained iteratively: each iteration generates games via MCTS-guided self-play, then updates θ to better predict game outcomes and the search-refined policy. In the original formulation, both players share a single policy and value head, and positions are canonicalized so the current player's pieces always appear as "friendly", which is natural for symmetric games where both sides face structurally identical decisions. In asymmetric games, where players differ in piece counts, objectives, and win conditions, this single head must learn two distinct evaluation functions, which can hinder learning efficiency and performance.

This work investigates whether $AlphaZero$'s self-play framework transfers to such a setting by applying it to Tablut, a historical board game played on a 9×9 board with 16 attackers against 8 defenders and a king. The attacker aims to capture the king; the defender aims to escort it to a corner (see Appendix A.1). The core algorithm transfers with one key modification—separate policy and value heads per player—but the asymmetric structure introduces training instabilities, such as catastrophic forgetting between roles, that were not reported in the original work.

2 Reproduction

The neural network architecture closely follows Silver et al. [2018], with the main difference being a reduced residual trunk of 8 blocks with 128 filters to match Tablut's lower complexity and more limited compute budget. The key modification is the use of separate policy and value heads for each player, $(\mathbf{p_a}, v_a, \mathbf{p_d}, v_d) = f_\theta(s)$, motivated by the fundamentally different objectives — king capture versus king escape — and the resulting asymmetric evaluation landscape. During MCTS, the head corresponding to the current player is selected, while the shared residual trunk learns common board features such as piece mobility and capture threats. Tablut's rook-like piece movement allows direct reuse of the action encoding from Silver et al. [2018]: each of the 81 squares has 32 directional planes (8 distances $\times$ 4 directions), yielding an action space of 2592 possible moves per position. See Appendix A.3 for state representation planes.

The system was implemented in JAX using Flax, Optax, Mctx, and Flashbax [Bradbury et al., 2018, DeepMind et al., 2020, Toledo et al., 2023]. To enable hardware-accelerated self-play, game environments were fully vectorized. While Koyamada et al. [2023] developed the Pgx library for this purpose, it does not natively support Tablut. Consequently, Tablut game logic was implemented from scratch by extending the Pgx base framework. Training was conducted on 2 NVIDIA H200 GPUs for 100 self-play iterations. In each iteration, generated states were stored in a replay buffer along with the MCTS-refined policy and the outcome reward of the corresponding game. The network was then trained on batches sampled from this buffer to predict game outcomes and policy distributions. MCTS

*Supervisor

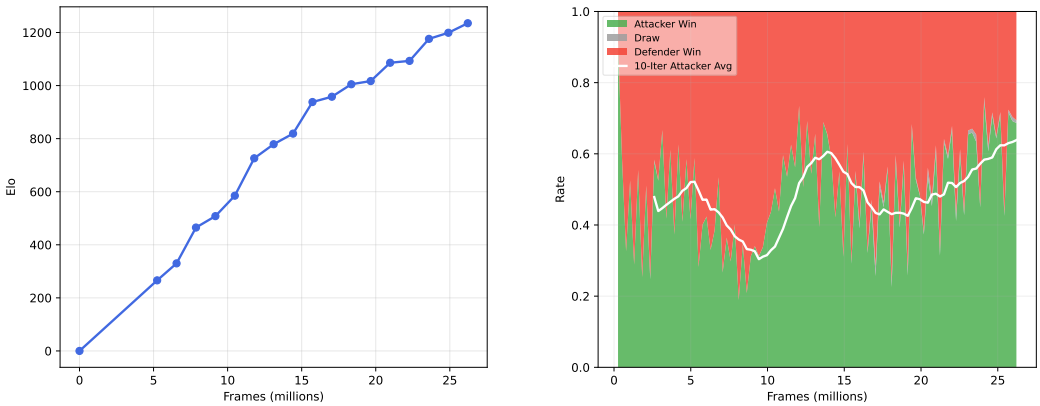
was performed using the $GumbelMuZero$ variant [Danihelka et al., 2022] with 128 simulations per move, which provides a more sample-efficient search than regular $PUCT$ under limited compute budget, while retaining the $AlphaZero$-style policy and value targets. The loss function combines mean squared error for value and cross-entropy for policy, as in $AlphaZero$. $AdamW$ with weight decay regularization was used for optimization. See Appendix A.3 for all hyper-parameters.

During training, performance against earlier checkpoints degraded — a phenomenon known as catastrophic forgetting in self-play. To stabilize training, C_4 data augmentation was applied (random 90° board rotations), and the replay buffer was increased from 8 to 16 self-play iterations. Inspired by Vinyals et al. [2019], 25% of training games were played against randomly sampled past checkpoints, with the current model alternating between attacker and defender roles. See Appendix A.2 for an ablation of these contributions.

Starting from the 20th iteration, the model was evaluated every 5 iterations against 4 opponents randomly sampled from a pool of up to 10 past checkpoints. The randomly initialized model (iteration 0) was retained as a fixed anchor. Following Silver et al. [2018], the $BayesElo$ program was used to calculate the Elo ratings across iterations. $BayesElo$ models draw rates and first-mover advantage, which is important for an asymmetric game where the inherent balance between sides is unknown.

3 Results and discussion

Over 100 iterations (26 million frames), the model reached a BayesElo rating of 1235 relative to the randomly initialized baseline (see Figure 1a). This indicates steady improvement, though the rating is only meaningful relative to this internal pool and not comparable to $AlphaZero$'s absolute performance on Chess or Go. Policy entropy decreased from 3.05 to 1.47 and the average number of pieces remaining at game end dropped from 22 to 15 (see Appendix A.4), reflecting increasingly focused and decisive play. Up to iteration 75, attackers and defenders achieved similar evaluation win rates of roughly 70–80% against the pool of past checkpoints when playing as that side. After that point, the defender's win rate declined to 52%, while the attacker's climbed to 86%, and in self-play the attacker's 10-iteration rolling average win rate reached about 65% by iteration 100 (see Figure 1b). This suggests either that the Tablut ruleset used favours the attacker or that defender strategies are harder to learn in this setting; a separate balance analysis was not performed to distinguish these explanations.



(a) Elo rating progression.

(b) Game outcomes (win rates).

Figure 1: Training metrics over 100 iterations. The model shows steady Elo growth and an increasing win rate for the attacker in Tablut.

The most challenging aspects of the reproduction were (i) down-scaling training from the much larger compute used by Silver et al. [2018] and (ii) mitigating catastrophic forgetting in self-play, which the original paper does not discuss in detail. Among the hyperparameters, the MCTS simulation count proved most critical: reducing it below 128 degraded both search quality and the resulting policy targets, while larger counts were impractical on two GPUs. Increasing the replay buffer size and using C_4 data augmentation both substantially improved stability.

References

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`.

Ivo Danihelka, Arthur Guez, Julian Schrittwieser, and David Silver. Policy improvement by planning with gumbel. In *International Conference on Learning Representations*, 2022.

DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Claudio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel, Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch, Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec, Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL `http://github.com/google-deepmind`.

Sotetsu Koyamada, Shinri Okano, Soichiro Nishimori, Yu Murata, Keigo Habara, Haruka Kita, and Shin Ishii. Pgx: Hardware-accelerated parallel game simulators for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 45716–45743, 2023.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419): 1140–1144, 2018.

Edan Toledo, Laurence Midgley, Donal Byrne, Callum Rhys Tilbury, Matthew Macfarlane, Cyprien Courtot, and Alexandre Laterre. Flashbax: Streamlining experience replay buffers for reinforcement learning with jax, 2023. URL `https://github.com/instadeepai/flashbax/`.

Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019.

A Appendix

A.1 Tablut rules

Tablut is a historic board game that was played in Northern Europe and its rules were first documented by Carl Linnaeus in 1732 in his diary. The original text, written in a mixture of Latin and Swedish, was subject to several problematic translations, leading to many different rule interpretations today. The rules used in this replication are the following:

- The game is played on a 9x9 board with 8 defenders, 1 king and 16 attackers. Regular pieces are also called taflmen.
- The king starts on the center square which is called the throne (see Figure 2).
- The pieces move like rooks in chess, and they cannot pass through other pieces.
- The objective of the defenders is to move the king to any of the corners of the board.
- The objective of the attackers is to capture the king.
- Any piece can be captured by sandwiching it between two enemy pieces. If a piece moves between two enemy pieces itself, then it is not captured.
- The king is captured like any other piece and it can also participate in captures.
- Only the king can step on the corners and the throne, but other pieces may pass through the throne.
- The throne and the corners are hostile squares, meaning that they can be used to capture pieces by both players. The throne becomes hostile to the defenders only when the king is not in the throne.
- If no captures have been made in the past 100 moves or it has not ended in 512 moves, then the game is a draw.
- If any board state appears for the third time during the game then the player who made the third repetition loses.
- If a player has no legal moves left, then they lose.

The game starts from a certain piece placement:

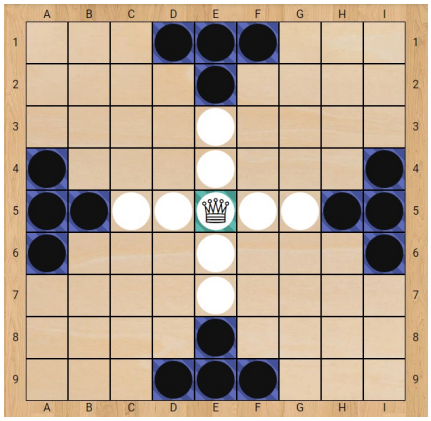


Figure 2: Tablut board's initial state

A.2 Elo Progression Ablation

Figure 3 compares the Elo progression across three training configurations, each building cumulatively on the previous one. The *Baseline* run uses the dual-head AlphaZero architecture with a replay buffer of 8 self-play iterations and no data augmentation. The + *Augmentation & Buffer* run adds C_4 data augmentation (random 90° board rotations) and increases the replay buffer from 8 to 16 self-play iterations ($16 \times 1{,}024 \times 256 \approx 4.2$M states). The + *Past-Iteration Self-Play* run further adds 25% of training games played against randomly sampled past checkpoints. Each technique yields a clear improvement in final Elo rating, with the full configuration reaching 1235.

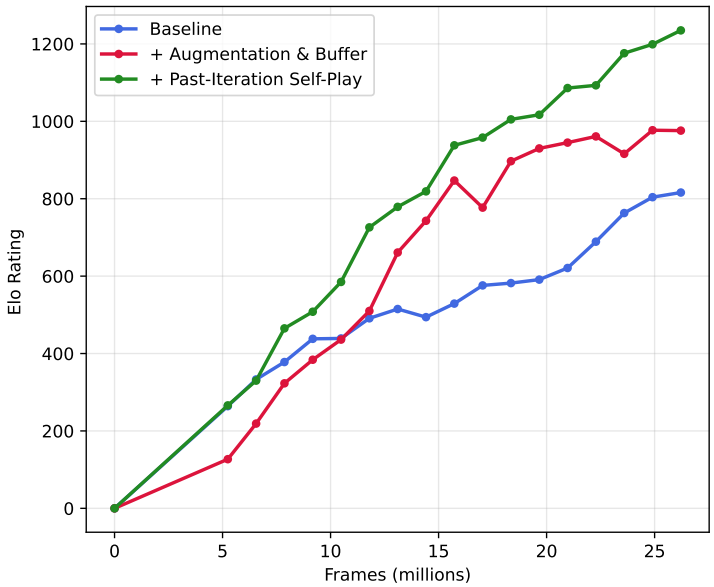


Figure 3: Elo progression across three cumulative training configurations. Each successive run adds one stabilization technique to the previous configuration.

A.3 State Representation and Hyperparameters

The input state s is a $9 \times 9 \times 43$ tensor, summarized in Table 1.

Table 1: State representation planes.

Feature	**Description**	**Planes**
	Per history step ($\times$ 8 steps)	
Friendly pieces	Current player's taflmen	1
Enemy pieces	Opponent's taflmen	1
King	King position	1
Repetition ≥ 1	Position seen before	1
Repetition ≥ 2	Position seen twice	1
	Auxiliary	
Player color	Current side to move	1
Total move count	Normalized step count	1
Half-move clock	Moves since last capture	1
Total		**43**

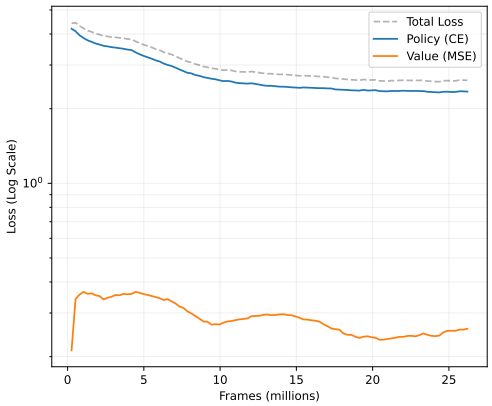
The hyperparameters used for the Tablut $AlphaZero$ replication are summarized in Table 2.

Table 2: Hyperparameters used for training and MCTS.

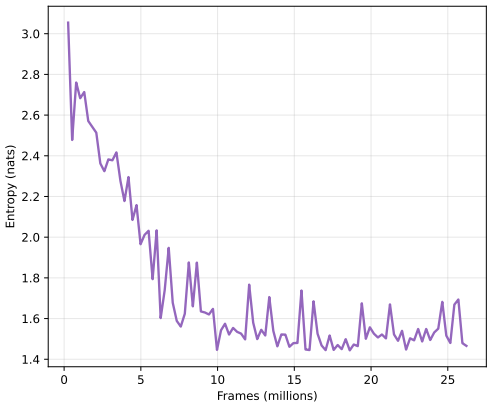
Hyperparameter	**Value**
Neural Network Architecture	
Residual blocks	8
Filters	128
Policy heads	2 (Attacker, Defender)
Value heads	2 (Attacker, Defender)
Training & Optimization	
Optimizer	$AdamW$
Weight decay	0.0001
Learning rate schedule	Cosine decay with warmup
Warmup steps	500
Peak learning rate	0.002
Minimum learning rate	0.00001
Total training steps	102,400
Batch size	512
Self-Play & MCTS	
Number of iterations	100
Parallel games per iteration	1024
Steps per game per iteration	256
MCTS simulations per move	128
Replay buffer size	4.2M states
Self-play vs. past versions	25%

A.4 Detailed Training Statistics

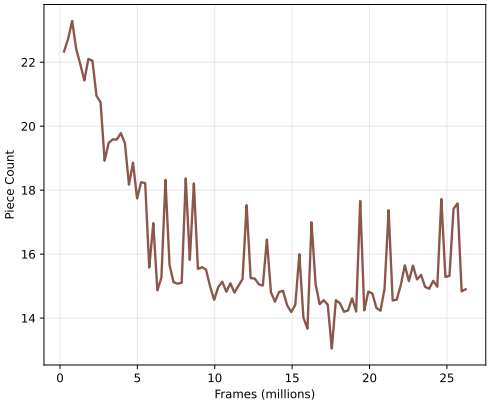
Figure 4 provides a granular view of the training process. The convergence of the policy and value losses (a) corresponds with the steady decrease in policy entropy (b), indicating the model is becoming more confident in its moves. Furthermore, the decrease in pieces remaining at the end of games (c) suggests more decisive play and efficient captures as training progresses.



(a) Combined Training Losses



(b) Policy Entropy



(c) Average Pieces Remaining

Figure 4: Detailed training metrics and game statistics recorded over 100 self-play iterations.